

Skill-5,6,7(16-09-2026)

15. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal Input:\n%s\n", input);

    printf("\nParsing quotes...\n");

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\') {
            printf("Single quote found at position %d\n", i);
        }
        else if (input[i] == '"') {
            printf("Double quote found at position %d\n", i);
        }
    }

    return 0;
}
```

Output:

```
manisha@LAPTOP-5SMR05RJ:~/SK$ nano skills.c
manisha@LAPTOP-5SMR05RJ:~/SK$ gcc skills.c -o skills
manisha@LAPTOP-5SMR05RJ:~/SK$ ./skills
Enter command: echo 'Manisha'

Original Input:
echo 'Manisha'

Parsing quotes...
Single quote found at position 5
Single quote found at position 12

manisha@LAPTOP-5SMR05RJ:~/SK$
```

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char *args[20];
        int i = 0;

        char *token = strtok(command, " ");

        while (token != NULL && i < 19) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("execvp failed");
        exit(1);
    }
    else {
        wait(NULL);
        printf("Command execution completed.\n");
    }

    return 0;
}

```

Output:

```
manisha@LAPTOP-5SMR05RJ:~/SK$ nano skills.c
manisha@LAPTOP-5SMR05RJ:~/SK$ gcc skills.c -o skills
manisha@LAPTOP-5SMR05RJ:~/SK$ ./skills
Enter command: ls
manisha  files    file1    fork.c    hello    output.txt  parent.c.save  partical  process.c  q1    skills  usernames.c
child    file.c   file1.c  fork2     hello.c   parent      parentid      partical.c  processid  q1.c   skills.c
child.c  file.txt fork     fork2.c   input.txt parent.c     parentid.c     process    processid.c q1.c.save  username

Command execution completed.
manisha@LAPTOP-5SMR05RJ:~/SK$ █
```

7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH:\n%s\n\n", path);

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child executing ls...\n");
        execlp("ls", "ls", NULL);

        perror("exec failed");
        exit(1);
    }
    else if (pid > 0) {
        int status;

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
            printf("Child exited with status %d\n",
                WEXITSTATUS(status));
    }
}

```

Output:

```
manisha@LAPTOP-5SMR05RJ:~/SK$ nano skills.c
manisha@LAPTOP-5SMR05RJ:~/SK$ gcc skills.c -o skills
manisha@LAPTOP-5SMR05RJ:~/SK$ ./skills

PATH:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/
lib/jvm/java-17-openjdk-amd64/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/
Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPower
Shell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/
JavaUpdate:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/
Git/cmd:/mnt/c/Program Files/nodejs:/mnt/c/Program Files (x86)/Intel/Intel(R) Management Engine
Components/iCLS:/mnt/c/Users/manisha/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/manisha/
AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/manisha/AppData/Roaming/npm:/snap/bin

Child executing ls...
manisha  file.c  file1  fork.c  hello  output.txt  parent.c.save  partical  process.c  q1  skills  usernames.c
child    file.c  file1.c  fork2  hello.c  parent  parentid  partical.c  processid  q1.c  skills.c
child.c  file.txt  fork  fork2.c  input.txt  parent.c  parentid.c  process  processid.c  q1.c.save  username

Child exited with status 0
manisha@LAPTOP-5SMR05RJ:~/SK$ █
```
